

React: Hooks, Custom Hooks y Context

Diego Muñoz

29 de septiembre de 2026

- Profundizar en los **hooks** de React.
- Entender su propósito y reglas.
- Crear **custom hooks** reutilizables.
- Introducir **Context** como solución al *prop drilling*.

- Función especial de React para usar **estado y ciclo de vida** en componentes funcionales.
- Todos comienzan con use.
- Ejemplos: `useState`, `useEffect`, `useContext`.

Antes de los hooks (React 16.8), solo los componentes de clase podían tener estado.

Reglas básicas de los Hooks

1. Solo se usan en el **nivel superior** del componente.
2. Solo se llaman dentro de **componentes React** u **otros hooks**.
3. Deben llamarse siempre en el mismo orden.

Esto permite que React asocie cada hook con su valor de estado correcto.

Donde NO va un hook

```
function Profile({ id }) {  
  if (id) {  
    const [user, setUser] = useState(null); // dentro de un if  
  }  
  for (const key of keys) {  
    useEffect(() => {}, []); // dentro de un bucle  
  }  
  const load = () => useState([]); // dentro de otra funcion  
}  
  
function formatName(user) {  
  return useState(""); // no es un componente  
}
```

useState: Estado local

- Hook más básico: permite crear un valor **reactivo**.
- Al cambiar el estado, React vuelve a renderizar el componente.

```
import { useState } from "react";
```

```
function Counter() {  
  const [count, setCount] = useState(0);  
  return (  
    <div>  
      <p>Valor: {count}</p>  
      <button onClick={() => setCount(count + 1)}>+</button>  
    </div>  
  );  
}
```

useState: Varios estados

- Puedes tener varios useState en el mismo componente.
- Cada uno mantiene su propio valor.

```
function Form() {  
  const [name, setName] = useState("");  
  const [age, setAge] = useState(0);  
  
  return (  
    <div>  
      <input value={name} onChange={e => setName(e.target.value)} />  
      <input value={age} onChange={e => setAge(+e.target.value)} />  
    </div>  
  );  
}
```

- Si el nuevo valor depende del anterior, usar **función de actualización**.

```
setCount(prev => prev + 1);
```

Esto evita errores cuando hay múltiples actualizaciones seguidas.

useEffect: Efectos secundarios

- Permite ejecutar código fuera del render: llamadas a APIs, timers, logs.
- Se ejecuta después de que React renderiza el componente.
- Puede devolver una función de limpieza que corre al desmontar.

```
import { useEffect } from "react";

useEffect(() => {
  console.log("Componente montado");
  return () => console.log("Desmontado");
}, []);
```

useEffect: Dependencias

El segundo argumento ([]) indica **cuándo** ejecutar el efecto.

Comportamiento de Dependencias

- [] Solo al montar componente
- [args] Cada vez que args cambien
- Sin [] Cada ciclo de render

```
useEffect(() => {  
  document.title = `Clicks: ${count}`;  
}, [count]);
```

El mismo fetch de la clase anterior, sin botón. El cuerpo del handler se mueve dentro del efecto y el arreglo vacío lo limita al montaje.

```
function Users() {  
  const [users, setUsers] = useState([]);  
  useEffect(() => {  
    fetch("https://jsonplaceholder.typicode.com/users")  
      .then(r => r.json()).then(setUsers);  
  }, []);  
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;  
}
```

- `useRef(x)` devuelve un objeto `{ current: x }`, y en cada render te devuelve el mismo objeto.
- Escribir en `.current` es JavaScript común. React ni se entera, por eso no rerenderiza.
- El atributo `ref={algo}` es otra cosa: le dice a React que ponga el nodo del DOM en `algo.current` al montar el elemento.
- Lo que se muestra en pantalla va en `useState`. Lo que no, en `useRef`.

useRef: estado que no repinta

Mismo componente, dos contadores. Lo único distinto es el hook.

```
function Contador() {  
  const [estado, setEstado] = useState(0);  
  const ref = useRef(0);  
  return (  
    <>  
      <p>Estado: {estado}</p>  
      <p>Ref: {ref.current}</p>  
      <button onClick={() => setEstado(estado + 1)}>+1 estado</button>  
      <button onClick={() => (ref.current += 1)}>+1 ref</button>  
    </>  
  );  
}
```

useRef: el nodo del DOM

Al poner `ref={videoRef}` en un elemento, React guarda ahí el nodo al montarlo. `videoRef.current` pasa a ser el elemento real, el mismo objeto que devolvería `document.querySelector`.

```
function Player() {  
  const videoRef = useRef(null);  
  return (  
    <>  
      <video ref={videoRef} src="/demo.mp4" width="320" />  
      <button onClick={() => videoRef.current.play()}>Reproducir</button>  
      <button onClick={() => videoRef.current.pause()}>Pausar</button>  
    </>  
  );  
}
```

useRef: los métodos son del elemento

- `play()` y `pause()` no vienen de `useRef`. Son del `<video>`, los define `HTMLMediaElement` en MDN.
- Un `<input>` no tiene `play()`. Tiene `focus()`, que define `HTMLElement`.
- Lo que puedes llamar depende del elemento que guardaste, no de React.
- Para ver qué tiene uno: `console.log(ref.current)` en el navegador.
- `useRef(null)` parte en `null`. Antes del montaje, o si el ref no quedó puesto en ningún elemento, llamar un método revienta.

useRef: lo que no se hace

Escribir el ref durante el render. React espera que el cuerpo del componente sea puro. El ref se toca en handlers o dentro de efectos.

```
function RenderCounter() {  
  const [text, setText] = useState("");  
  const renders = useRef(0);  
  renders.current += 1;                                // durante el render  
  return (  
    <>  
      <input value={text} onChange={e => setText(e.target.value)} />  
      <p>Renders: {renders.current}</p>  
    </>  
  );  
}
```

- Alternativa a useState cuando el estado crece o admite varias acciones.
- La lógica del cambio vive en una función pura, fuera del componente.

```
function reducer(state, action) {  
  switch (action.type) {  
    case "increment": return { count: state.count + 1 };  
    case "decrement": return { count: state.count - 1 };  
    case "reset": return { count: 0 };  
    default: return state;  
  }  
}
```

useReducer: Consumirlo

dispatch envía la acción y React calcula el estado nuevo con el reducer.

```
function Counter() {  
  const [state, dispatch] = useReducer(reducer, { count: 0 });  
  return (  
    <>  
      <p>Total: {state.count}</p>  
      <button onClick={() => dispatch({ type: "increment" })}>+</button>  
      <button onClick={() => dispatch({ type: "decrement" })}>-</button>  
      <button onClick={() => dispatch({ type: "reset" })}>Reset</button>  
    </>  
  );  
}
```

useMemo y useCallback

- `useMemo` guarda el resultado de un cálculo y lo reusa mientras las dependencias no cambien.
- `useCallback` hace lo mismo con una función, conserva su referencia.
- React Compiler memoiza solo desde su versión 1.0, estable en octubre de 2025. Son un concepto a entender, no una práctica obligatoria.

```
const found = useMemo(  
  () => NAMES.filter(n => n.includes(query)),  
  [query]  
);
```

React trae 17 hooks integrados. El curso usa cinco y el resto cubre casos específicos.

- **Estado:** `useState`, `useReducer`
- **Contexto:** `useContext`
- **Referencias:** `useRef`, `useImperativeHandle`
- **Efectos:** `useEffect`, `useLayoutEffect`, `useInsertionEffect`, `useEffectEvent`
- **Rendimiento:** `useMemo`, `useCallback`, `useTransition`, `useDeferredValue`
- **Otros:** `useDebugValue`, `useId`, `useSyncExternalStore`, `useActionState`

Custom Hooks: Lógica reusable

- Un **custom hook** encapsula lógica que usa otros hooks.
- Permite compartir comportamiento entre componentes.

Custom Hooks: Ejemplo

```
function useToggle(initial = false) {  
  const [value, setValue] = useState(initial);  
  const toggle = () => setValue(v => !v);  
  return [value, toggle];  
}  
  
function App() {  
  const [open, toggleOpen] = useToggle();  
  return (  
    <>  
      <button onClick={toggleOpen}> {open ? "Cerrar" : "Abrir"} </button>  
      {open && <p>Contenido visible</p>}  
    </>  
  );  
}
```

Custom Hooks: useFetch

El patrón de tres estados de la clase anterior, encapsulado.

```
function useFetch(url) {  
  const [data, setData] = useState(null);  
  const [isLoading, setIsLoading] = useState(true);  
  const [error, setError] = useState(null);  
  useEffect(() => {  
    fetch(url)  
      .then(r => r.json()).then(setData)  
      .catch(e => setError(e.message))  
      .finally(() => setIsLoading(false));  
  }, [url]);  
  return { data, isLoading, error };  
}
```

useFetch: Consumirlo

```
function Users() {  
  const { data, isLoading, error } = useFetch(  
    "https://jsonplaceholder.typicode.com/users"  
  );  
  
  if (isLoading) return <p>Cargando...</p>;  
  if (error) return <p>Error: {error}</p>;  
  
  return <ul>{data.map(u => <li key={u.id}>{u.name}</li>)}</ul>;  
}
```

Cualquier componente que necesite datos reusa el hook.

useContext: Evitar prop drilling

- Context permite **compartir datos globales** sin pasar props manualmente.
- Ideal para tema, idioma o usuario.

```
import { createContext } from "react";  
export const ThemeContext = createContext();
```

```
function ThemeProvider({ children }) {  
  const [dark, setDark] = useState(false);  
  
  return (  
    <ThemeContext.Provider value={{ dark, setDark }}>  
      {children}  
    </ThemeContext.Provider>  
  );  
}
```

Consumir el contexto

```
function ToggleTheme() {  
  const { dark, setDark } = useContext(ThemeContext);  
  return (  
    <button onClick={() => setDark(!dark)}>  
      {dark ? "Modo claro" : "Modo oscuro"}  
    </button>  
  );  
}  
  
function App() {  
  return (  
    <ThemeProvider> <ToggleTheme /> </ThemeProvider>  
  );  
}
```

Estructura recomendada

```
src/  
  components/  
  hooks/  
    useToggle.js  
    useFetch.js  
  context/  
    ThemeContext.jsx  
  App.jsx
```

Organiza el código por **función y responsabilidad**.

- `useState`: manejar estado local.
- `useEffect`: ejecutar efectos secundarios.
- `useRef`: acceder al DOM y guardar valores sin re-render.
- `useReducer`: estado con varias acciones.
- `useMemo`: reusar un cálculo entre renders.
- Custom hooks: encapsular lógica reutilizable.
- `Context`: compartir datos globales.
- `useContext`: consumir el contexto.

¿Tienes dudas? ¡Hablemos!